UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CURSO DE TECNOLOGIA EM SISTEMAS PARA INTERNET

DANILO AUGUSTO PINOTTI DE MELLO

SOLUÇÃO PARA MONITORAMENTO AMBIENTE UTILIZANDO ARDUINO

PROJETO DE TRABALHO DE CONCLUSÃO DE CURSO

GUARAPUAVA

2016

DANILO AUGUSTO PINOTTI DE MELLO

SOLUÇÃO PARA MONITORAMENTO AMBIENTE UTILIZANDO ARDUINO

Projeto de Trabalho de Conclusão de Curso apresentada à disciplina de Trabalho de Conclusão de Curso I do Curso de Tecnologia em Sistemas para Internet da Universidade Tecnológica Federal do Paraná como requisito integral para aprovação.

Orientador: Prof. Fábio Leandro Janiszevski

Co-orientador: Prof. Ms. Paulo Henrique Soares

GUARAPUAVA

2016

RESUMO

MELLO, Danilo A. P.. SOLUÇÃO PARA MONITORAMENTO AMBIENTE UTILIZANDO ARDUINO. 40 f. Projeto de Trabalho de Conclusão de Curso – Curso de Tecnologia em Sistemas para Internet, Universidade Tecnológica Federal do Paraná. Guarapuava, 2016.

Existem diversos ambientes em que é estritamente necessário o monitoramento dos mesmos, como por exemplo, CPDs, câmaras frias e granjas. Atualmente é possível encontrar sistemas desenvolvidos unicamente para esta finalidade, porém nem sempre estes estão ao alcance de todos, devido a fatores como, custo de aquisição e implantação, ou disponibilidade no mercado nacional. O objetivo deste projeto é desenvolver um sistema utilizando a plataforma Arduino juntamente com sensores, de forma que o sistema alerte devidamente um usuário, ou um grupo de usuários em casos de anomalias no ambiente e que seja capaz de fornecer informações sobre cada ambiente monitorado por uma interface Web.

Palavras-chave: Arduino, sensores, MQTT, pub-sub, monitoramento ambiente

ABSTRACT

MELLO, Danilo A. P.. Environment monitor solution with Arduino. 40 f. Projeto de Trabalho de Conclusão de Curso – Curso de Tecnologia em Sistemas para Internet, Universidade Tecnológica Federal do Paraná. Guarapuava, 2016.

There are many environments in which it is strictly necessary monitoring thereof, such as computer rooms, cold rooms and farms. Currently it is possible find systems developed exclusively for this purpose, but not always they are available to everyone, because of factors such as cost of acquisition and implementation, or availability in the national market. The objective of this project is to develop a system using the Arduino platform with sensors, in a way that the system properly warn a user or group of users in cases of anomalies in the environment and is capable of providing information about each environment monitored by a Web interface.

Keywords: Arduino, sensors, MQTT, pub-sub, environment monitor

LISTA DE FIGURAS

Figura 1 – Poseidon 2 4002 10
Figura 2 – Interface Web do Poseidon 2 11
Figura 3 – Exemplo de monitoramento realizado pelo PRTG 11
Figura 4 – Modulo DNT24 12
Figura 5 – Ranking das linguagens mais utilizadas 15
Figura 6 – Arduino Mega 19
Figura 7 – Módulo Ethernet para Arduino 20
Figura 8 – Shield Ethernet para Arduino 20
Figura 9 – Exemplos de sensores 21
Figura 10 – Sensor LDR 22
Figura 11 – Gráfico da resistência de um LDR em função da luminosidade 22
Figura 12 – Sensor LM35 23
Figura 13 – Sensor DHT11 24
Figura 14 – Representação da arquitetura Pub/Sub 26
Figura 15 – Exemplo de Bot do Telegram. 27
Figura 16 – Cenário proposto 30
Figura 17 – Arduino Software 32
Figura 18 – Arquitetura do sistema 33

LISTA DE TABELAS

TABELA 1 – Tabela de comparação entre os trabalhos correlatos 12
TABELA 2 – Características do Arduino Mega 19
TABELA 3 – Cronograma 36

LISTA DE SIGLAS

API	Application Programming Interface
CPD	Centro de Processamento de Dados
CSS	Cascading Style Sheets
DB2	Database 2
DNS	Domain Name Server
HTML	Hypertext Markup Language
HTTP	Hyper Text Transfer Protocol
HTTPS	Hyper Text Transfer Protocol Secure
IBM	International Business Machines
IDE	Integrated Development Environment
IP	Internet Protocol
LCD	Liquid Crystal Display
LDR	Light Dependent Resistor
LED	Light Emitter Diode
M2M	Machine-to-Machine
MIT	Massachusetts Institute of Technology
MQTT	Message Queueing Telemetry Transport
PHP	Hypertext Preprocessor
PRTG	Paessler Router Traffic Grapher
QoS	Quality of Service
SGBD	Sistema Gerenciador de Banco de Dados
SMS	Short Message Service
SNMP	Simple Network Management Protocol
SPI	Serial Peripheral Interface
SQL	Structed Query Language
SSL	Secure Socket Layer
TCP	Transmission Control Protocol
TI	Tecnologia da Informação
UTFPR	Universidade Tecnológica Federal do Paraná
XML	EXtensible Markup Language

SUMÁRIO

1 INTRODUÇÃO **8**
1.1 OBJETIVOS 9
1.1.1 Objetivo Geral 9
1.1.2 Objetivos Específicos 9
1.2 DIFERENCIAL TECNOLÓGICO 9
2 RESENHA LITERÁRIA **10**
2.1 ESTADO DA ARTE 10
2.2 FUNDAMENTAÇÃO TEÓRICA 13
2.2.1 HTML 13
2.2.2 CSS 13
2.2.3 PHP 14
2.2.4 Laravel - Framework PHP 15
2.2.5 SQL 16
2.2.6 MySQL 16
2.2.7 Arduino 17
2.2.7.1 Arduino Mega 18
2.2.7.2 Módulos 19
2.2.7.3 Shields 20
2.2.8 Sensores 21
2.2.8.1 LDR 21
2.2.8.2 LM35 23
2.2.8.3 DHT11 23
2.2.9 Protocolo MQTT 24
2.2.10Padrão Publish/Subscribe 25
2.2.11Telegram 26
3 METODOLOGIA **28**
4 DESENVOLVIMENTO **30**
4.1 CENÁRIO PROPOSTO 30
4.2 MATERIAIS UTILIZADOS 31
4.2.1 Hardware 31
4.2.2 Software 31
4.2.2.1 Arduino Software 31
4.2.2.2 ActiveMQ Apollo 32
4.3 SISTEMA 33
4.3.1 Servidor Central 33
4.3.2 Módulo com Arduino 34
5 CONSIDERAÇÕES FINAIS **35**
5.1 CONTRIBUIÇÃO DO TRABALHO 35
5.2 DIFICULDADES ENCONTRADAS 35
5.3 LIÇÕES APRENDIDAS 35
6 CRONOGRAMA **36**

REFERÊNCIAS 37

1 INTRODUÇÃO

A cada dia que passa, milhares de dispositivos novos são conectados à Internet. Segundo Evans (2011), entre 2003 e 2010, a quantidade de dispositivos conectados à Internet ultrapassaram a quantidade de pessoas no mundo. Pesquisas apontam que até o ano de 2020, a quantidade de dispositivos serão equivalentes a 6,5 vezes a quantidade de pessoas, não relacionada somente à dispositivos pessoais. Atualmente, cerca de 75% dos dispositivos conectados à Internet são para usos pessoais e, até 2020, este número tende a cair para 25%. Toda essa quantidade de dispositivos conectados pode ser relacionada com a Internet das Coisas. O termo Internet das Coisas é utilizado para definir o uso de sistemas embarcados conectados à Internet que possuem a capacidade de se comunicarem com outros dispositivos, serviços ou pessoas em escala global (MUKHOPADHYAY, 2014).

De acordo com Cunha (2007) , sistema embarcado é a capacidade computacional aplicada em um circuito integrado, equipamento ou sistema. Estes sistemas, por sua vez, são criados para exercer apenas uma funcionalidade durante sua vida útil, como fornos de micro-ondas, roteadores, televisores, dentre outros. Também são exemplos de sistemas embarcados sensores com sistemas operacionais próprios, sistemas expansíveis que trabalham com protocolos específicos de monitoramento ou até mesmo ares condicionados de precisão. Além disso, sistemas embarcados são utilizados como dispositivos para monitoramento ambiente, os quais verificam constantemente as condições atuais de temperatura e umidade para tomarem as devidas providências. Um exemplo deste tipo de dispositivo é o uso de alarmes de incêndio que, quando disparados, notificam o corpo de bombeiros automaticamente para requisitar o envio de uma equipe profissional no local para averiguar a situação.

No decorrer do presente trabalho, será desenvolvido um sistema que integre sensores através de um Arduino. Esta solução deve ser capaz de realizar o monitoramento de dados de determinados ambientes, de forma que a mesma seja capaz de notificar usuários ou grupos de usuários do sistema de acordo com gatilhos pré-programados como por exemplo, uma variação brusca de temperatura, umidade e luminosidade.

1.1 OBJETIVOS

Serão apresentados nessa seção o objetivo geral e os objetivos específicos do projeto.

1.1.1 OBJETIVO GERAL

Propor uma solução para monitoramento ambiente utilizando Arduino e sensores, de forma que notifique usuários do sistema em casos de anomalias na temperatura, luminosidade e umidade.

1.1.2 OBJETIVOS ESPECÍFICOS

- Implementar comunicação cliente-servidor com o uso do protocolo MQTT (*Message Queueing Telemetry Transport*).
- Implementar em Arduinos, sensores que verifiquem as condições do ambiente como luz, temperatura e umidade.
- Desenvolver um sistema Web que integre os Arduinos com sensores.
- Integrar ferramentas de notificações no sistema Web.
- Monitorar o ambiente real e confirmar com dados dos sensores que serão implantados no CPD (Centro de Processamento de Dados) da UTFPR (Universidade Tecnológica Federal do Paraná) campus Guarapuava para validação do sistema desenvolvido.

1.2 DIFERENCIAL TECNOLÓGICO

Destaca-se que no projeto será possível monitorar mais de um ambiente ao mesmo tempo apenas incluindo outro módulo (constituído por um Arduino, sensores e *shield* Ethernet) no sistema e instalando-o fisicamente.

2 RESENHA LITERÁRIA

Nesta seção serão abordados o estado da arte (seção 2.1) bem como apresentar resoluções semelhantes do problema. Também será apresentada a fundamentação teórica (seção 2.2), com as tecnologias utilizadas no projeto.

2.1 ESTADO DA ARTE

Com a necessidade cada vez maior de um monitoramento rigoroso e preciso em CPDs, granjas e ambientes do gênero, atualmente, é possível encontrar no mercado dispositivos embarcados com a finalidade de monitorá-los. Uma das soluções encontradas no mercado é o Poseidon 2 4002 o qual é apresentado na Figura 1, que, por sua vez, é capaz de utilizar mais de 28 sensores (varia de acordo com o modelo) para cada ambiente instalado, podendo detectar fumaça, água, gás, vibrações além de monitorar temperatura e umidade. O dispositivo em questão pode ser configurado para enviar e-mail ou SMS (*Short Message Service*) aos usuários registrados, bem como ser monitorado via SNMP (*Simple Network Management Protocol*) por alguma aplicação específica como o Zabbix, Cacti ou Nagios (HW-GROUP, 2016a).

Figura 1: Poseidon 2 4002
Fonte: HW-Group (2016).

Além dos recursos citados, o Poseidon fornece uma interface Web para configuração do equipamento e dados resultantes do monitoramento do CPD. A Figura 2 mostra a tela inicial da interface Web de um Poseidon 2 4002.

Por outro lado, também existem sistemas capazes de monitorar recursos internos de um

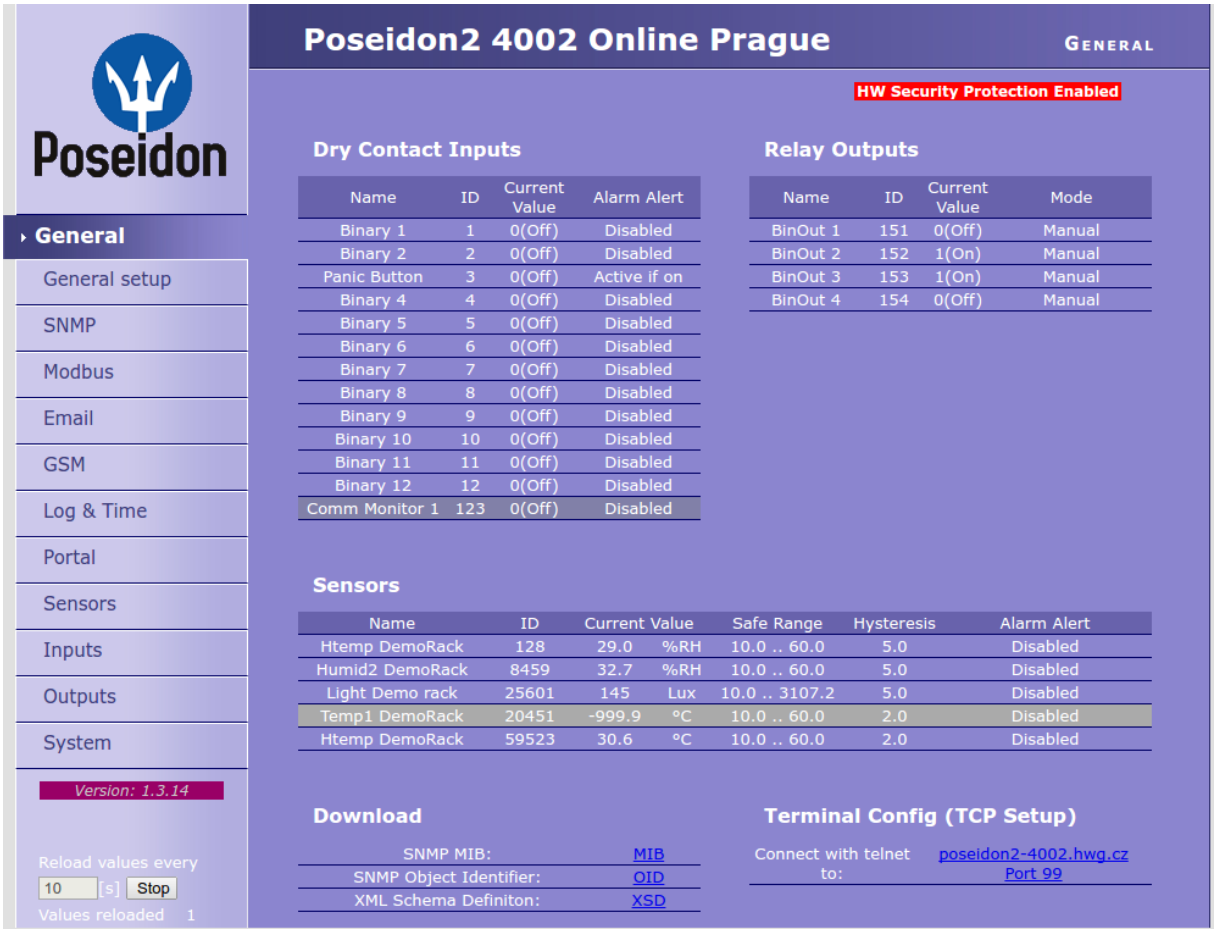

Figura 2: Interface Web do Poseidon 2
Fonte: HW-Group (2016).

CPD, como o funcionamento de um equipamento específico ou de serviços. Um exemplo desse tipo de sistema é o PRTG, desenvolvido pela empresa alemã Paessler, que tem seu uso gratuito para monitoramento de até 100 sensores (equipamentos e serviços). O acompanhamento de alguns indicadores, pode auxiliar uma ação proativa dos profissionais de TI (Tecnologia da Informação), ajudando-os a descobrir problemas e evitá-los antes mesmo de terem acontecido (XANDó, 2012).

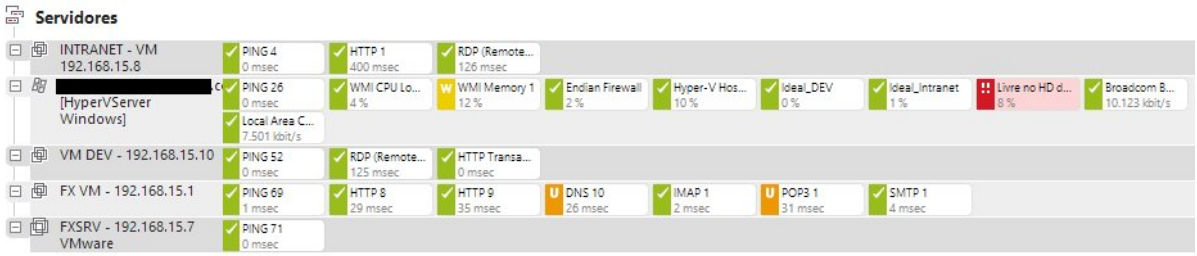

Figura 3: Exemplo de monitoramento realizado pelo PRTG
Fonte: Xandó (2012).

Com a utilização do PRTG, ao qual é demonstrado na Figura 3, é possível obter informações como: disponibilidade, uso de memória e processamento, resposta de requisição HTTP (*Hyper Text Transfer Protocol*) (no caso de servidores Web), resolução de nomes (no caso de servidor DNS) (*Domain Name Server*), dentre outros.

Outro produto relacionado é o DNT24. Este é utilizado para medição, controle e manutenção de condições ambientais de granjas tais como temperatura de incubadoras, umidade, níveis de água para as aves, dentre outros. É de alta importância manter o ambiente confortável e livre de estresse para as aves, para que, dessa forma, a granja consiga uma produção otimizada de ovos e menor prejuízo em caso de falhas em equipamentos (RFM, 2016).

A Figura 4 representa um módulo DNT24 ao lado de uma moeda para poder comparar seu tamanho.

Figura 4: Modulo DNT24
Fonte: Imagem adaptada de RFM (2016).

Para o funcionamento, sensores e atuadores são conectados aos módulos DNT24 aos quais enviam, via ondas de rádio, as informações do ambiente, periodicamente, para um controlador base. Fica a cargo da aplicação do cliente escolher de qual forma estes dados serão tratados.

A tabela 1 abaixo lista a característica e críticas dos trabalhos recém-citados.

Tabela 1: Tabela de comparação entre os trabalhos correlatos

Trabalhos	Poseidon 2 4002	PRTG	DNT24
Características	Produto voltado para CPDs; Grande variedade de sensores; Múltiplos sensores trabalhando simultaneamente; Notificações via SMS, e-mail, entre outras.	Produto voltado para CPDs; Monitoramento de equipamentos; Múltiplos sensores trabalhando simultaneamente; Monitoramento de serviços.	Produto voltado para uso geral; Monitoramento de ambiente; Múltiplos sensores trabalhando simultaneamente; Conexão sem fio entre sensores e a base.
Críticas	Cada aparelho monitora apenas um ambiente.	Não faz monitoramento do ambiente físico em questão.	Não provê interface gráfica para o usuário. Fica a cargo do cliente tratar as informações recebidas.

Fonte: Autoria própria.

2.2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo são abordadas as tecnologias utilizadas no projeto.

2.2.1 HTML

O HTML, abreviação de *Hypertext Markup Language*, é uma linguagem de marcação utilizada no desenvolvimento de sites. Criada em 1991 por Tim Berberners-Lee, esta surgiu junto com o protocolo HTTP. Foi a primeira linguagem de nível mundial, porém, não foi a única. É possível criar páginas para a Web utilizando HTML mesclado com diversas outras linguagens, como o PHP (*Hypertext Preprocessor*), Ruby, dentre outros (PACIEVITCH, 2016).

A criação de códigos HTML é baseada em marcações conhecidas como *tags*, em que cada uma tem uma função diferente. Arquivos HTML podem ser criados e editados usando simples editores de texto e devem ser salvos com a extensão .htm ou .html.

A cada nova versão lançada a linguagem se torna mais fácil de ser interpretada e utilizada. Para o projeto será utilizada a versão 5 que possui, nativamente, elementos para criação de gráficos (canvas e svg), elementos multimídia (áudio e vídeo) e novos elementos pré-definidos para formulários (W3SCHOOLS, 2016b).

2.2.2 CSS

O CSS, abreviação de *Cascading Style Sheets*, é uma linguagem utilizada para descrever estilos em documentos, sendo eles HTML ou XML (*EXtensible Markup Language*) (W3SCHOOLS, 2016a). É de responsabilidade do CSS definir o que será renderizado na tela ou não e de qual forma cada elemento será apresentado.

O CSS trabalha com o efeito cascata em que é estabelecida uma prioridade na aplicação das regras de estilo aos elementos. A prioridade é dada da seguinte forma (considerando o primeiro item como prioridade mais alta): estilos declarados no elemento, estilos declarados em uma folha de estilização local (no próprio arquivo HTML) e estilos declarados em folhas de estilização externas (arquivos com extensão .css). Outro fator que determina a prioridade é a ordem em que são declarados nos arquivos, quanto mais “próximo” do elemento e mais para baixo na declaração, maior a prioridade (SILVA, 2003).

2.2.3 PHP

Criado em 1994 por Rasmus Lerdorf, o PHP, inicialmente significando Personal Home Page, tinha como proposta utilizar os programas desenvolvidos pelo próprio criador em suas paginas Web. Com o passar do tempo e com a aceitação das pessoas, a linguagem foi tomando forma até se tornar o que conhecemos hoje e com o significado de *Hypertext Preprocessor*, ou, pré-processador de hipertexto (ALVAREZ, 2004).

O PHP é uma linguagem de programação interpretada, amplamente utilizada para desenvolvimento Web, que devido a sua capacidade de se misturar ao HTML, torna mais fácil a criação de páginas dinâmicas (SOARES, 2010).

Reyes (2009) cita várias finalidades para o uso do PHP, dentre elas estão:

- *E-Commerce* através do Magento, Zen Cart, Shopify, dentre outros;
- Fóruns através do phpBB, vBulletin, PunBB, dentre outros;
- Aplicações para o Facebook;
- Sistemas para manuseio de conteúdo, como Wordpress, Joomla e Drupal.

De acordo com a Fuctura (2016), são vantagens em utilizar o PHP:

- A facilidade para aprender a linguagem;
- A praticidade para acessar SGBDs (Sistema Gerenciador de Banco de Dados), como o MySQL e o Oracle;
- O fato de ser multiplataforma permite que você use aplicações feitas em PHP em computadores com diversos sistemas operacionais diferentes;
- Outro destaque do PHP é que ele é *open-source*, ou seja, têm o código fonte aberto.

Também deve ser levado em consideração as desvantagens em utilizar a linguagem (FUCTURA, 2016):

- A compatibilidade entre versões não é totalmente confiável, ou seja, não há uma padronização de determinados recursos entre as versões;
- A segurança deixa a desejar quando comparada a um Servlet do Java EE, por exemplo.

De acordo com um *ranking* de linguagens mais utilizadas detalhado por Tiobe (2016) através de motores de busca, o PHP está em 6º lugar como linguagem de programação mais utilizada no mundo em 2016, como apresentado na Figura 5.

May 2016	May 2015	Change	Programming Language	Ratings	Change
1	1		Java	20.956%	+4.09%
2	2		C	13.223%	-3.62%
3	3		C++	6.698%	-1.18%
4	5	^	C#	4.481%	-0.78%
5	6	^	Python	3.789%	+0.06%
6	**9**	^	**PHP**	**2.992%**	**+0.27%**
7	7		JavaScript	2.340%	-0.79%
8	15	^^	Ruby	2.338%	+1.07%
9	11	^	Perl	2.326%	+0.51%
10	8	v	Visual Basic .NET	2.325%	-0.64%

Figura 5: Ranking das linguagens mais utilizadas
Fonte: Tiobe (2016).

Este projeto usará o PHP na versão 5.5.9 ou superior como linguagem padrão para o sistema Web desenvolvido. Os motivos para esta escolha são a sua ampla comunidade de desenvolvedores e o suporte multiplataforma.

2.2.4 LARAVEL - FRAMEWORK PHP

Criado em 2011 por Taylor Otwell, o Laravel é um *framework* PHP *open-source* sobre a licença MIT (*Massachusetts Institute of Technology*), voltado para aplicações Web. Em cinco anos de existência, o Laravel se tornou um dos mais conhecidos *Frameworks* PHP e apontado como um dos melhores e mais completos (SCHIMIGUEL, 2016).

O Laravel utiliza uma *Engine* de *template* chamada de *Blade* para a criação da interface gráfica. Esta *Engine* traz uma série de ferramentas para auxiliar na criação rápida de interfaces agradáveis e funcionais, além da redução de códigos duplicados. (ADRIEL, 2015).

A comunicação com o Banco de Dados é feita através do Eloquent ORM. Esta comunicação é facilitada para o desenvolvedor visto que a ferramenta traz diversos recursos para o gerenciamento das informações no Banco de Dados (ADRIEL, 2015).

Quando comparado com outros *frameworks*, o Laravel tem como desvantagem o desempenho, o tamanho relativamente maior, o tempo elevado de *debug* em determinadas ocasiões e a comunidade relativamente menor (AGRIYA, 2015), porém, a facilidade de aprendizado, os

recursos, a documentação e a sua estabilidade fazem com que o Laravel seja um dos *frameworks* PHP mais utilizados no mundo e também no presente projeto.

2.2.5 SQL

A Linguagem Estruturada de Consultas (SQL, *Structed Query Language*) foi desenvolvida para ser capaz de criar e dar manutenção em bases de dados relacionais além de permitir o gerenciamento dos dados nessas bases (SHELDON, 2009).

Atualmente, boa parte dos SGBDs utilizam o SQL como linguagem padrão para o acesso às bases de dados. Dentre eles podem ser citados o DB2 da IBM, Oracle da Oracle Corporation, SQL Server da Microsoft, e MySQL da Oracle Corporation (XDSOFTWARE, 2016).

Através do SQL, o utilizador torna-se capaz de criar, apagar e modificar base de dados além de poder inserir, apagar, atualizar e buscar informações dos mesmos.

Dentre as utilidades do SQL citadas em TutorialsPoint (2016) do SQL podem ser destacadas que:

- Permite o usuário acessar dados em bases de dados relacionais;
- Permite a manipulação dos dados definidos na base de dados;
- Permite a criação e alteração de tabelas e bases de dados;
- Permite a embarcação com outras linguagens com através da utilização módulos, pré-compiladores e bibliotecas;
- Permite a criação de *views*, *stored procedures* e *functions* nas bases de dados.
- Permite aos usuários definirem permissões de acesso às tabelas, *procedures* e *views*.

O projeto irá usar o SQL como linguagem padrão para comunicação com o SGBD definido, o MySQL.

2.2.6 MYSQL

O MySQL é um sistema gerenciador de banco de dados relacional, *open-source* e é um dos mais utilizados e conhecidos no cenário mundial. Criado na década de 90 por David Axmark, Allan Larsson e Michael Monty Widenius. O MySQL acabou se tornando conhecido

devido ao seu desempenho ao gerenciar dados e, cada vez mais vem sendo utilizado em projetos e sistemas (MILANI, 2007).

Grandes empresas como Youtube, PayPal, Google, Facebook, Cisco, entre outras, utilizam o MySQL em seus sistemas (MYSQL, 2016).

De acordo com Gilmore (2008) a popularidade do MySQL foi conquistada devido aos seguintes motivos:

- Flexibilidade: Independentemente do sistema operacional que está sendo utilizado, o MySQL têm grandes chances de atender à necessidade.
- Capacidade: Desde o início do projeto, o foco do MySQL é o desempenho. Com as atualizações e facilidades empregadas, a equipe de desenvolvimento ainda mantém um compromisso em relação à sua velocidade.
- Recursos SQL a nível de empresa: A partir da versão 5.0 o MySQL traz suporte a views, subqueries e stored procedures.
- Indexação e busca *full-text*: São recursos que permitem maior velocidade em buscas de forma a trazer os resultados por ordem de relevância.
- *Query caching*: Permite que o MySQL armazene em memória os resultados das buscas junto com a *querie SELECT* (recurso utilizado para buscar informações na base de dados).
- Replicação: Permite que um banco de dados seja replicado em um ou mais servidores MySQL.
- Segurança: O MySQL permite, através de configurações, definir permissões de acesso para diversos recursos e configurações para praticamente tudo que o SGBD é capaz de disponibilizar.
- Opções flexíveis para licença além da sua comunidade de usuários altamente ativa.

Por ser um SGBD livre, eficaz e com uma comunidade relativamente ativa, o MySQL foi escolhido para ser utilizado no projeto para armazenar dados no sistema.

2.2.7 ARDUINO

O Arduino é uma plataforma eletrônica *open-source* de prototipagem baseada em hardware e software flexíveis e fáceis de usar (ARDUINO, 2016g). São chamados de projetos *open-*

source aqueles em que qualquer pessoa pode modificar e distribuir, tornando-o desta forma, totalmente acessível ao público (OPENSOURCE, 2016).

Segundo McRoberts (2010), o Arduino é um pequeno computador onde pode ser programado para processar entradas e saídas entre o dispositivo e os componentes externos conectados a ele, como por exemplo, ligar um ventilador que está conectado a um relé quando a temperatura do ambiente for superior a 30°C.

Existem diversos modelos de Arduino disponíveis para a utilização, dentre eles estão:

- Arduino Uno: dado como um dos mais simples devido à sua baixa capacidade de armazenamento e processamento. É ideal para quem está iniciando na área de eletrônica e programação (ARDUINO, 2016e).
- Arduino Mega: possui uma quantidade maior de portas analógicas e digitais e maior capacidade de processamento e armazenamento em relação ao modelo Uno. Este, por sua vez é utilizado em projetos maiores que demandam maior capacidade do dispositivo (ARDUINO, 2016b).
- Arduino Yún: placa projetada para Internet das Coisas, onde visa principalmente a conectividade entre dispositivos. Combina a capacidade de um Linux com a facilidade de um Arduino (ARDUINO, 2016f).

A programação do Arduino é feita através de um ambiente de desenvolvimento próprio, onde utiliza-se de uma linguagem de programação também própria, semelhante ao C/C++.

Arduinos normalmente são utilizados em projetos para automação industrial, ou residencial, e podem ser conectados a displays, LEDs (*Light Emitter Diode*), sensores, motores, botões e qualquer outro dispositivo que possa ser controlado ou que emita dados.

2.2.7.1 ARDUINO MEGA

O Arduino Mega, apresentado na Figura 6, foi projetado para executar projetos mais complexos e que exijam mais capacidade de processamento, armazenamento e quantidade de pinos analógicos e digitais. Este possui 54 pinos digitais de entrada e saída e mais 16 pinos analógicos (ARDUINO, 2016b).

A tabela 2 apresentada contém informações adicionais sobre o Arduino Mega.

Os motivos pelos quais foi escolhido o Arduino Mega para o projeto são: baixo custo,

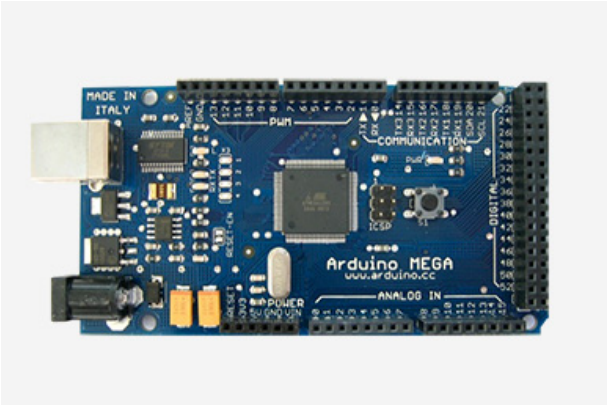

Figura 6: Arduino Mega
Fonte: Arduino (2016).

Tabela 2: Características do Arduino Mega

Tensão de operação	5V
Tensão de entrada (recomendado)	7-12V
Tensão de entrada (limites)	6-20V
Memória flash	128KB com 4KB utilizado no bootloader
SRAM	8KB
EEPROM	4KB
Velocidade de Clock	16MHz
Quantidade de portas digitais	54 (onde 15 fornecem saída PWM)
Quantidade de portas analógicas	16
Corrente contínua por pino	40mA
Corrente contínua do pino 3.3V	50mA

Fonte: Adaptada de Arduino (2016).

capacidade para alimentação externa, portas analógicas e digitais suficientes para possíveis expansões futuras.

2.2.7.2 MÓDULOS

Módulos são dispositivos que permitem aumentar a capacidade do Arduino. Estes são conectados nas portas digitais e/ou analógicas separadamente e através de fios (LIMA, 2013). Os módulos ocupam as portas disponíveis no Arduino, ou seja, terá que ser destinadas portas analógicas e/ou digitais exclusivamente para o módulo de acordo com a necessidade do mesmo.

Existem módulos para as mais diversas funcionalidades, por exemplo, módulo relé, módulo Bluetooth, módulo Ethernet (Figura 7), dentre outros.

Figura 7: Módulo Ethernet para Arduino
Fonte: Filipeflop (2016).

2.2.7.3 SHIELDS

Shields são placas que podem ser conectadas por cima do Arduino e que estendem sua capacidade (ARDUINO, 2016c). Existe uma variedade grande de *shields* para Arduino, como *shield* com tela LCD (*Liquid Crystal Display*), *shield* para leitura de cartões microSD, dentre outros.

A principal vantagem em utilizar *shields* é que, ao contrário dos módulos, mesmo com elas instaladas, as portas digitais e analógicas nem sempre são ocupadas. Existem modelos de shields que conseguem manter todas as portas (*shield* Ethernet por exemplo) e também modelos que ocupam todas as portas (alguns modelos de *shield* LCD por exemplo).

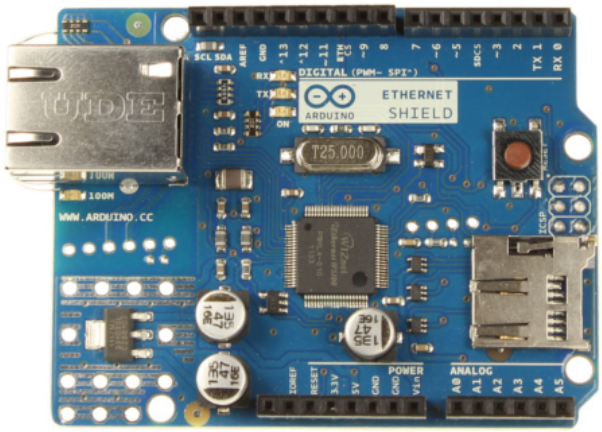

Figura 8: Shield Ethernet para Arduino
Fonte: Arduino (2016).

Para o presente projeto, será utilizado uma *shield* Ethernet (Figura 8) que serve para conectar um Arduino em uma rede local ou à Internet através de um conector RJ45. O *shield*

em questão trabalhará sob o protocolo TCP/IP com velocidade de conexão de 10/100Mb e será alimentado com 5V pela própria placa Arduino. Sua comunicação com o Arduino é dada através da porta SPI (*Serial Peripheral Interface*), desta forma, não consumindo as outras portas da placa (ARDUINO, 2016a).

2.2.8 SENSORES

Sensores são dispositivos eletroeletrônicos que têm a propriedade de transformar uma grandeza física em sinais elétricos através de uma ou mais propriedades do material de que é feito o sensor (STEFFENS, 2016).

Existem sensores para as mais diversas necessidades como medir temperatura, medir a concentração de gás carbônico, medir a luminosidade, detectar som, detectar movimento, dentre outros. A Figura 9 apresenta alguns exemplos de sensores.

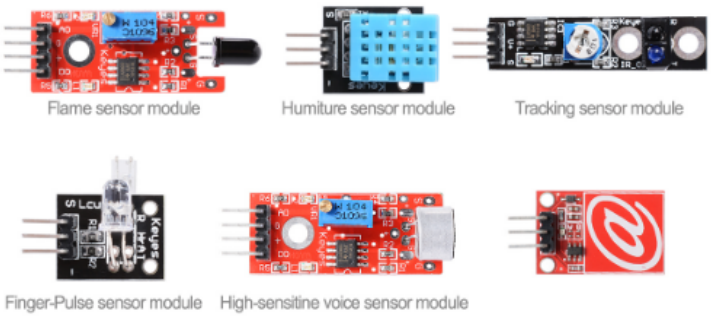

Figura 9: Exemplos de sensores
Fonte: Adaptada de AliExpress (2016).

Todos os sensores podem ser classificados, basicamente, em dois tipos, podendo eles serem sensores analógicos ou sensores digitais. Esta divisão é feita de acordo com a resposta do sensor à variação da condição. Sensores analógicos são os que retornam, mesmo que em uma faixa de valores definida, resultados infinitos intermediários a esta faixa, já os sensores digitais, retornam apenas dois resultados, sendo ele verdadeiro ou falso (LOW ou HIGH no Arduino) (PARSKO, 2006).

2.2.8.1 LDR

O LDR (*Light Dependent Resistor*) é um componente eletrônico capaz de variar sua resistência elétrica em função da luminosidade incidente sobre ele. A resistência deste sensor é inversamente proporcional à quantidade de luz sobre ele, ou seja, quanto mais luz, menos resistência. O cálculo da resistência de um LDR é dado obedecendo a seguinte equação:

$$R = C.L.a$$

De forma que L representa a luminosidade em Lux, “C” e “a” constantes individuais de acordo com o material utilizado e do processo de fabricação (BRITTO, 2003).

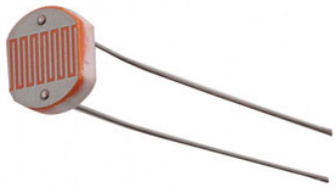

Figura 10: Sensor LDR
Fonte: NinjaGecko (2016).

Um LDR comum (Figura 10) tem aproximadamente a resistência de 5.000 ohms sob a luz do dia e 20.000.000 ohms na escuridão (KITRONIK, 2014). A Figura 11 apresenta o gráfico da relação entre a intensidade da luz e a resistência de um LDR.

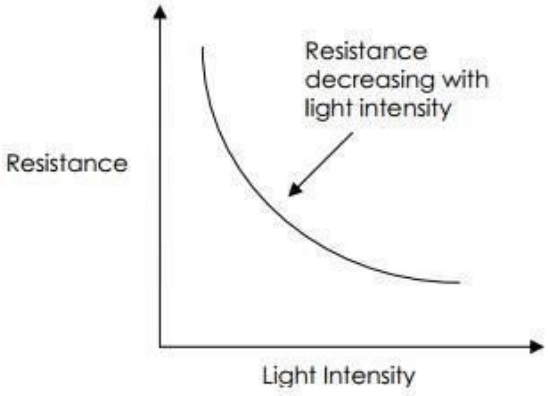

Figura 11: Gráfico da resistência de um LDR em função da luminosidade
Fonte: Eletrical4U (2016).

Por ter um preço baixo e estrutura simples, o LDR é utilizado, por exemplo, em postes de luz, despertadores, alarmes, contagem de pacotes em esteiras, dentre outros (ELETRICAL4U, 2016).

Além disso, o sensor também é utilizado em circuitos para detectar claridade em ambientes abertos ou fechados, podendo desta forma, detectar em ambientes sem iluminação um possível incêndio, a abertura de uma porta ou a passagem de pessoas em determinadas ocasiões. Pelas características recém citadas, este sensor será um dos componentes presentes neste projeto.

2.2.8.2 LM35

Criado pela National Semiconductor, a série LM35 é formada por dispositivos com circuitos integrados de precisão com o objetivo de obter a temperatura de ambientes. A temperatura é obtida através da tensão de saída do sensor e, esta é diretamente proporcional à temperatura do ambiente onde o sensor está localizado (NATIONALSEMICONDUCTOR, 2000).

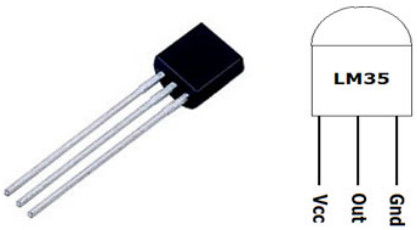

Figura 12: Sensor LM35
Fonte: Saddam (2015).

A Figura 12 apresenta um LM35 (lado esquerdo) junto com sua representação gráfica (lado direito).

De acordo com o *Datasheet* do sensor, o LM35 é capaz de trabalhar com tensão de entrada entre 4 e 30 volts e trabalha corretamente em uma faixa de temperatura de -55°C até 150°C.

A precisão aprimorada quando comparada ao DHT11 e o custo relativamente baixo deste sensor são motivos para usá-lo no projeto.

2.2.8.3 DHT11

O DHT11 (repesentado na Figura 13) é um sensor digital capaz de obter a temperatura e umidade do ambiente em que está presente. O DHT11 não é um sensor de alta precisão, logo, não é recomendado para ambientes de alto risco (ARDUINOECIA, 2013).

Este sensor trabalha com a tensão elétrica entre 3.3V e 5.5V, com a faixa de umidade entre 20 e 90% e faixa de temperatura entre 0 e 50°C. A precisão do sensor é de 2°C e 5% de umidade. Das características do DHT11, pode ser destacado o baixo consumo de energia, sinal digital calibrado, o fato de não necessitar de mais componentes para poder usar e às bibliotecas disponíveis para suporte (SUNROM, 2012).

O DHT11 é importante para o projeto para a captação da umidade do ambiente, visto que é um dos sensores de umidade mais baratos encontrados no mercado ,e , também, devido a facilidade de uso na aplicação.

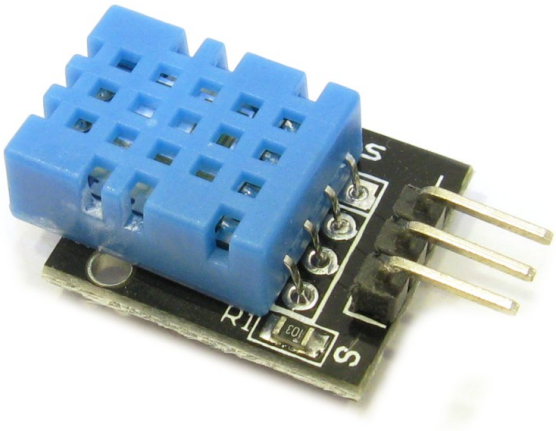

Figura 13: Sensor DHT11
Fonte: Modtronix (2016).

2.2.9 PROTOCOLO MQTT

Criado por Andy Stanford-Clark, com suas origens na IBM, o MQTT, *Message Queueing Telemetry Transport*, é um protocolo de comunicação M2M (*Machine-to-Machine*) para dispositivos dentro de um domínio *publish/subscribe* projetado para Internet das Coisas, desta forma, é caracterizado pela sua comunicação rápida e leve (BROWN, 2014).

Os padrões do protocolo se encontram atualmente na versão 3.1. De acordo com suas especificações, ele foi projetado para ser *open-source*, simples, leve e fácil de implementar. Essas características tornam o protocolo ideal para uso em redes congestionadas, grandes, de baixa velocidade ou não seguras. Além disso, o protocolo é ideal para ser utilizado em dispositivos embarcados cuja memória e capacidade de processamento são limitadas (EUROTECH, 2010).

O protocolo utiliza do padrão de envio de mensagens *publish/subscribe* onde torna possível o envio de mensagens de um para vários dispositivos (um-para-muitos). Também são características do protocolo (LEE et al., 2013):

- Conexão com a rede através do TCP/IP;
- Com o cabeçalho de apenas 2 bytes, o MQTT provê trocas minimizadas de informações resultando em menos tráfego na rede;
- Assíncrono com três níveis de qualidade de serviço (QoS):
  - "No máximo uma vez": nível utilizado onde as mensagens são enviadas apenas uma vez pela rede, desta forma, correndo o risco de se perderem no meio do caminho. Um exemplo de uso seria o de envio de dados de sensores onde, caso perca algum

dos valores, logo em seguida será enviado outro;

- "Ao menos uma vez": nível onde as mesmas mensagens são enviadas mais de uma vez pela rede. Usando esta configuração, pode acontecer da mensagem chegar duplicada no destinatário;
- "Exatamente uma vez": nível em que garante a chegada da mensagem ao destinatário. Utilizada quando a informação é crítica e precisa chegar sem erros ou duplicações ao destinatário, exemplo, uma transação monetária.

O *Facebook Messenger* é um exemplo de produto criado com base no MQTT (LEE et al., 2013). Este protocolo será o responsável no projeto pela comunicação entre o cliente e o servidor devido a simplicidade e desempenho em redes congestionadas ou lentas.

2.2.10 PADRÃO PUBLISH/SUBSCRIBE

O princípio do modelo de comunicação *Publish/Subscribe* (pub/sub) está em os dispositivos registrarem seus interesses nos tipos de informações para que possam recebê-las, sempre que possível, dos dispositivos fornecedores. O processo de registrar tal interesse é chamado de subscrição, portanto, o dispositivo interessado logo é chamado de subscritor (*subscribers*). Por outro lado, os dispositivos que irão fornecer as mensagens passarão por um processo chamado de publicação, Estes são chamados de publicadores (*publishers*) (HUNKELER; TRUONG; STANFORD-CLARK, 2008).

De acordo com Hunkeler e Truong (2008) existem três tipos de sistemas pub/sub:

- Baseado em tópicos: neste tipo de sistema, todos os tópicos já estão definidos do inicío ao final do processo de desenvolvimento.
- Baseado em tipos: os subscritores declaram qual tipo de dado tem interesse (dados de temperatura por exemplo).
- Baseado no conteúdo: este é o tipo mais versátil dentre os outros. O subscritor declara qual o conteúdo que desejará receber (pode haver dados de sensores de luz e temperatura ao mesmo tempo).

Um sistema *publish/subscribe* é implementado sobre um agente de eventos conhecido como *broker*. *Brokers* são responsáveis em rotear eventos entre os publicadores e os subscritores (BANAVAR et al., 1999). A Figura 14 faz uma representação da arquitetura do padrão em questão.

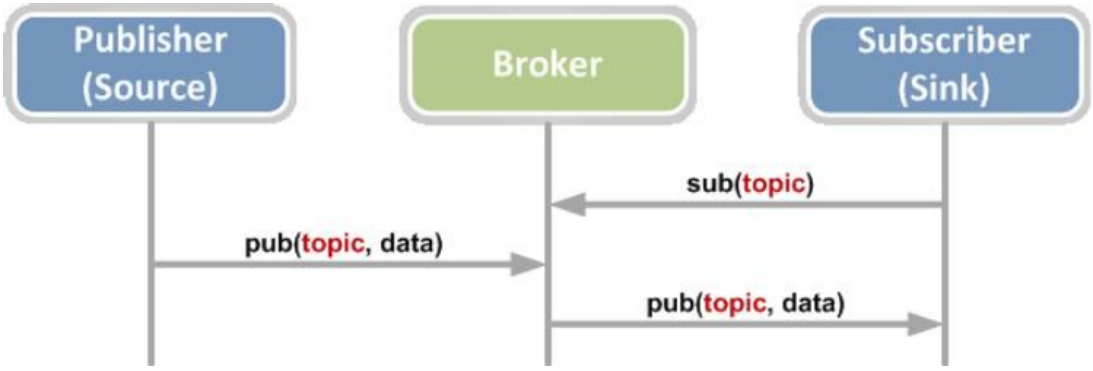

Figura 14: Representação da arquitetura Pub/Sub
Fonte: Adaptada de Kannan (2015).

O protocolo MQTT, utilizado no projeto, é implementado no padrão *publish/subscribe*, logo, a comunicação entre o módulo Arduino e o servidor irão respeitar as especificações recém descritas.

2.2.11 TELEGRAM

Telegram é um aplicativo *open-source* de mensagem para desktop e mobile baseado em nuvem, fundado em 2013, cujo foco é sua segurança e velocidade. É possível utilizar o Telegram em vários dispositivos ao mesmo tempo, e sua sincronização de mensagens é dita como ideal, independentemente da quantidade de clientes conectados a uma mesma conta (TELEGRAM, 2016b).

A encriptação das mensagens no Telegram, seja texto, foto, áudio, vídeo, dentre outros, pode ser feita em duas camadas. A primeira delas é entre o servidor e o cliente, utilizada em chats privados ou em grupos de conversa, e, a segunda, além da camada de cliente-servidor, é feita também uma outra entre remetente e destinatário (cliente para cliente) (TELEGRAM, 2016b).

Além da segurança e velocidade do Telegram, é possível a utilização e criação de Bots através da API (*Application Programming Interface*) aberta de Bots.

De acordo com a equipe de desenvolvimento do Telegram (2016), Bots são aplicações de terceiros que funcionam dentro do Telegram. Usuários podem interagir com estes Bots através de mensagens e linhas de comando. Bots são controlados através de requisições HTTPS (*Hyper Text Transfer Protocol Secure*) para a API de Bot. A Figura 15 apresenta um exemplo de Bot cuja função programada para ele é de enviar materiais de estudo para os usuários poderem aprender línguas novas.

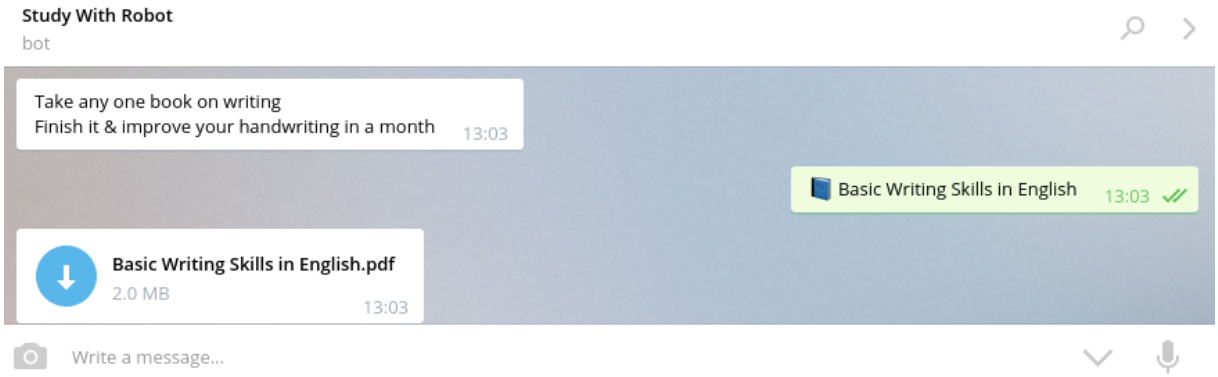

Figura 15: Exemplo de Bot do Telegram.
Fonte: Autoria própria

O Telegram será um dos meios utilizados para alertar os usuários de possíveis incidentes nos ambientes monitorados através de um Bot criado para tal finalidade. Também será possível interagir com o sistema através do Bot, como por exemplo, requisitar o estado atual de cada ambiente.

3 METODOLOGIA

Nesta seção, são apresentadas as etapas que serão realizadas a fim de alcançar os objetivos do projeto.

- Definição das tecnologias;
- Estudo das tecnologias definidas;
- Elaboração de um protótipo do módulo com Arduino;
- Desenvolvimento do sistema Web e algoritmo do Arduino;
- Realização de testes gerais do sistema;
- Realização de testes em um CPD real.

Logo em seguida, uma descrição de cada etapa citada da metodologia:

Definição das tecnologias: inicialmente, serão definidas as tecnologias a serem utilizadas no projeto. O desenvolvimento será realizado dentro dos limites impostos por estas tecnologias. Portanto, será importante escolher quais melhor atendem as necessidades especificadas. Para a escolha das tecnologias serão avaliados critérios como a eficácia, a comunidade relacionada e o custo de aquisição e manutenção de cada uma.

Estudo das tecnologias definidas: esta etapa é constituída por pesquisas, na Internet e em livros, referentes às tecnologias utilizadas no projeto, até que seja estabelecida uma base de conhecimento necessária para dar início ao processo de desenvolvimento.

Elaboração de um protótipo do módulo com Arduino: será montado o protótipo do módulo que, na versão final do projeto, deverá estar presente em cada ambiente a ser monitorado. O módulo é constituído por um Arduino Mega, uma *shield* Ethernet e os sensores: DHT11, LM35 e LDR.

Desenvolvimento do sistema Web e algoritmo do Arduino: mediante o Arduino IDE (*Integrated Development Environment*) e um editor de texto voltado para programação, será desenvolvido o sistema Web, responsável em prover uma interface para o usuário. Também será feito o algoritmo do Arduino, este responsável por obter os dados dos sensores e enviar para o servidor Web.

Realização de testes gerais do sistema: em paralelo ao desenvolvimento serão feitos testes para cada nova etapa do sistema desenvolvida. Os testes realizados nessa etapa, inicialmente, irão ser compostos de entradas de informações aleatórias com finalidade de simular um ambiente real, e, juntamente a este processo, o acompanhamento do comportamento do sistema, com a finalidade de verificar a estabilidade do mesmo, bem como efetuar correções quando necessário.

Realização de testes em um CPD real: com a versão inicial do projeto funcionando, o sistema será implantado no CPD da UTFPR câmpus Guarapuava e passará por observação. Nesta etapa, o sistema já estará trabalhando com dados reais e, é de importante função, a comparação dos dados processados por ele com os dados reais obtidos através de outras ferramentas do gênero, como termômetros e câmeras.

4 DESENVOLVIMENTO

Nessa seção será descrito as etapas e características do desenvolvimento do projeto.

4.1 CENÁRIO PROPOSTO

O sistema será desenvolvido, tomando como base para o projeto, o CPD (Centro de Processamento de Dados) da UTFPR câmpus Guarapuava, onde possui dois aparelhos de ares condicionados e é definido, pela equipe responsável pelo CPD, um limite máximo de temperatura de 25ºC. Tal limite foi imposto de acordo com as normas descritas em TIA-942 onde a temperatura ideal de um CPD deve ser entre 20 e 25ºC.

O ambiente em questão provê todos os recursos necessários para a realização do projeto, tal como, energia elétrica, espaço físico para instalação do módulo e cada sensor dele, acesso à rede interna e acesso à Internet.

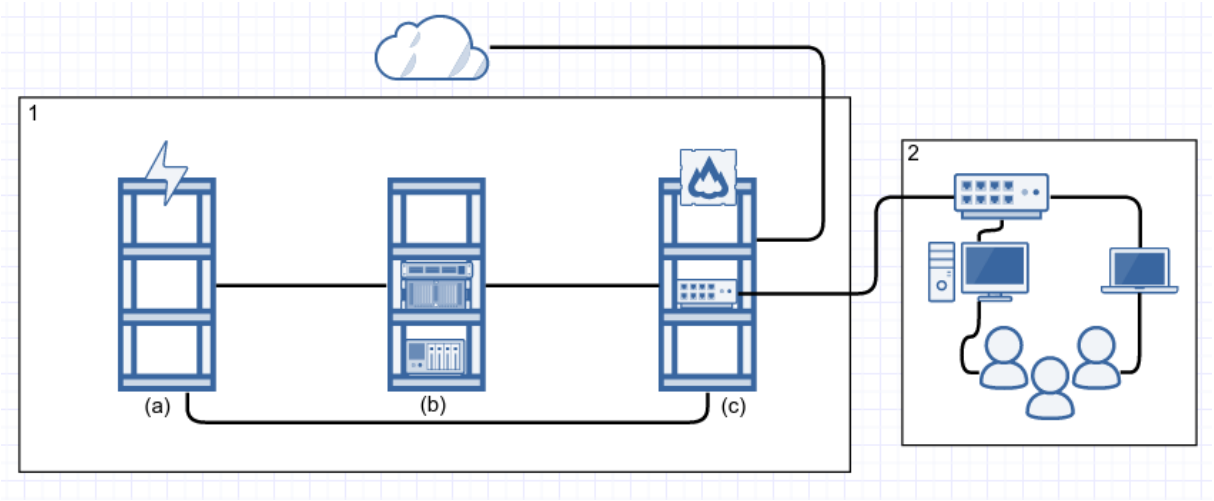

Figura 16: Cenário proposto
Fonte: Autoria própria

A Figura 16 retrata o cenário descrito. O ambiente representado pelo número 1 na Figura descreve o CPD onde (a) indica o rack de energia, (b) o rack de servidores, onde ficará o

servidor central do sistema, e (c) o rack de rede. A instalação do sistema será feita no ambiente 1 e, através da rede, o módulo Arduino será diretamente conectado à rede através de um *switch* localizado no rack de rede (c). O ambiente 2 representa o local onde se encontra equipe de TI, responsável pelo CPD e pelos serviços de rede oferecidos pelo câmpus.

4.2 MATERIAIS UTILIZADOS

Nessa seção serão apresentados os materiais utilizados para o desenvolvimento do projeto.

4.2.1 HARDWARE

Para o desenvolvimento do projeto será necessário:

- Um Arduino Mega: Será utilizado para integrar todos os sensores e enviar as informações para o servidor;
- Uma *shield* Ethernet: Será utilizada para dar ao Arduino Mega acesso à rede local através de um conector RJ45 e sob o protocolo TCP/IP;
- Sensores: Sensores DHT11, LM35 e LDR serão responsáveis por captar dados sobre o ambiente em tempo real;
- Estrutura de rede: Será necessário ter uma rede local funcional com rotas definidas entre o Arduino e o servidor Web para que possa haver comunicação entre eles;
- Servidor Web: O servidor Web será usado para armazenar os dados obtidos e prover uma interface para os usuários.

4.2.2 SOFTWARE

Nessa seção serão descritas as ferramentas utilizadas para o desenvolvimento e funcionamento do sistema.

4.2.2.1 ARDUINO SOFTWARE

O Arduino Software (apresentado na Figura 17) é uma IDE *open-source* que torna mais fácil a escrita de códigos e o envio do mesmo para o Arduino. Desenvolvido sob a linguagem

Java, este é multiplataforma, ou seja, é possível utilizá-lo em diversos sistemas operacionais, como Mac OS X, Windows e Linux (ARDUINO, 2016d).

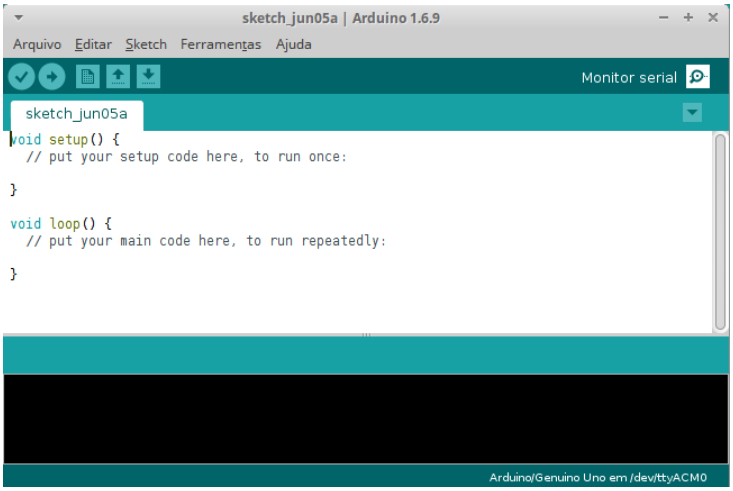

Figura 17: Arduino Software
Fonte: Autoria própria

A utilização do Arduino Software se torna essencial no projeto pois, através dele, é possível escrever o código, fazer verificação de sintaxe, compilar o código escrito, enviar o algoritmo compilado para o Arduino e fazer monitoramento Serial. Será utilizada a versão mais recente desta IDE, atualmente, encontrada na 1.6.9.

A linguagem de programação utilizada para programar em um Arduino foi feita baseada em C/C++ trazendo algumas modificações e alguns recursos nativos essenciais, como a função utilizada para ler valores em portas digitais e analógicas também como constantes nativas como HIGH e LOW ou INPUT e OUTPUT (ARDUINO, 2016h).

A IDE também conta, nativamente, com uma gama de exemplos de uso de componentes e *shields* que podem ser utilizados como base para desenvolvimento para outros projetos.

4.2.2.2 ACTIVEMQ APOLLO

Diversos protocolos de comunicação necessitam de uma camada intermediária entre o dispositivo remetente e o destinatário para que seja possível o envio e recebimento das mensagens. Essas camadas intermediárias são chamadas de *brokers*.

O ActiveMQ Apollo é um projeto *open-source*, confiável e rápido que funciona como *broker* para diversos protocolos, como o MQTT, SSL, STOMP, etc. (ACTIVEMQ, 2016).

Será utilizado no projeto o protocolo MQTT para a comunicação entre máquinas (M2M). O Apollo atua como *broker* da conexão entre clientes MQTT que é um protocolo *open-*

source utilizado em dispositivos com recursos limitados e redes muito congestionadas usando um domínio *publish-subscribe* (ACTIVEMQ, 2016).

4.3 SISTEMA

A solução consistirá em duas partes. A primeira se trata de um servidor central que reunirá informações dos ambientes monitorados, e irá disparar notificações em ocasiões citadas por um usuário administrador.

A segunda parte consiste em um Arduino conectado a sensores de luminosidade, sensores de temperatura e sensores de umidade juntamente com um *shield* Ethernet conectado à Internet ou a uma rede local, em que será responsável em enviar todas as informações recolhidas do ambiente para o servidor. Para cada ambiente monitorado deverá haver um módulo Arduino.

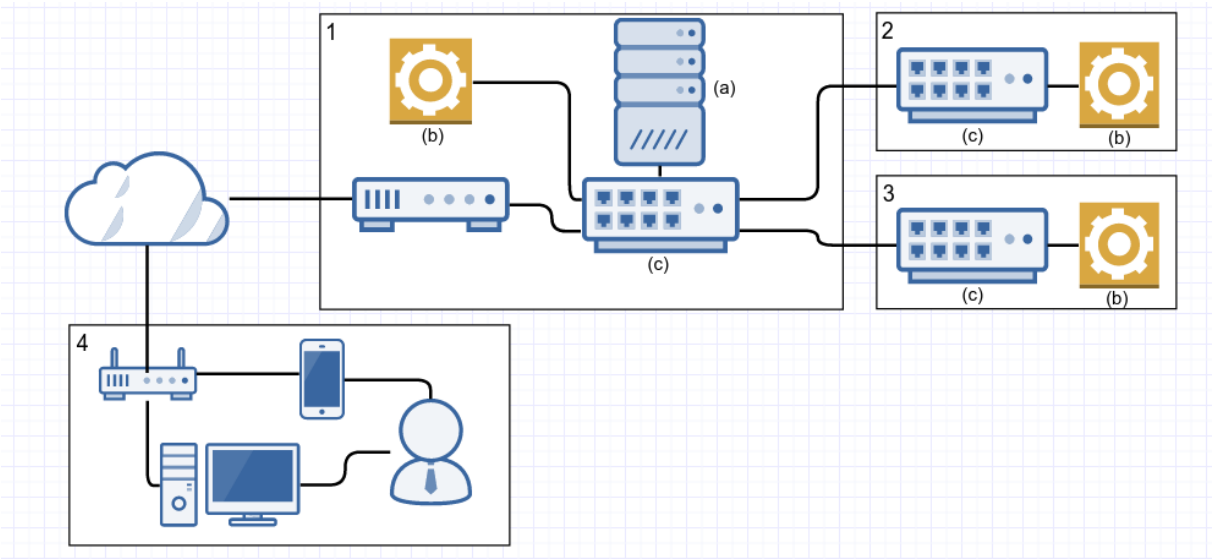

Figura 18: Arquitetura do sistema
Fonte: Autoria própria

A Figura 18 retrata a arquitetura física do sistema, onde possui 4 salas (cada uma dentro de um retângulo isolado). A sala representada pelo número 1 é onde o ficará o servidor central (a) e, junto a ele, um módulo Arduino (b) para monitorar este ambiente. As salas 2 e 3 representam dois ambientes distintos que irão ser monitorados pelo sistema, com acesso à rede através de um *switch* (c). A sala 4 representa a casa ou qualquer outro lugar que o usuário esteja, que, via Internet, poderá acessar interface Web do projeto.

4.3.1 SERVIDOR CENTRAL

Cabe ao servidor central:

1. Fazer conexão com todos os módulos de forma que seja capaz de enviar e receber dados de cada um;
2. Prover uma interface Web para os usuários do sistema em que seja possível realizar configurações do sistema e obter informações sobre os ambientes monitorados;
3. Notificar usuários do sistema sobre inconformidades programadas do ambiente (gatilhos) através de e-mail ou Telegram;
4. Permitir a criação de gatilhos de notificação. Um exemplo de gatilho seria o envio de um e-mail para todos os usuários em caso da temperatura ultrapassar 25ºC ou em caso da luminosidade ultrapassar um limite imposto para o ambiente;
5. Guardar dados provenientes dos módulos Arduino.

Para o desenvolvimento deste servidor serão seguidas as seguintes etapas:

- Projetar um banco de dados para armazenar informações necessárias ao sistema;
- Utilizar o protocolo MQTT para comunicação entre cliente e servidor;
- Desenvolver o sistema Web utilizando o Laravel PHP *framework*;
- Criar um sistema capaz de monitorar mais de um ambiente simultaneamente.

4.3.2 MÓDULO COM ARDUINO

Cabe ao módulo Arduino:

1. Obter dados do ambiente em que está contido;
2. Ter conexão com o servidor central;
3. Enviar dados recolhidos para o servidor central.

O desenvolvimento do módulo é constituído por:

- Montar o circuito com o Arduino, *shield* Ethernet e sensores;
- Implementar a comunicação com o servidor;
- Programar a captura das informações do ambiente e envio delas para o servidor.

Cada sensor não ficará ligado diretamente no Arduino. Eles poderão ser conectados e desconectados com facilidade devido ao uso de plugs no projeto.

5 CONSIDERAÇÕES FINAIS

Nesta seção apresenta-se as dificuldades encontradas, as contribuições e as lições aprendidas no decorrer do trabalho.

5.1 CONTRIBUIÇÃO DO TRABALHO

Após a concretização desse projeto o sistema poderá ser utilizado em qualquer CPD dentro da UTFPR de forma gratuita.

Inicialmente será implantado no câmpus Guarapuava devido ao fato de dar suporte e fornecer o ambiente para testes durante o desenvolvimento.

5.2 DIFICULDADES ENCONTRADAS

As principais dificuldades encontradas no momento da escrita deste projeto foram em relação a fazer o levantamento das desvantagens e as empresas que utilizam cada tecnologia.

5.3 LIÇÕES APRENDIDAS

Durante a escrita do projeto foram aprendidos detalhes importantes em relação às tecnologias que serão utilizadas. Exemplos que se destacam são:

- Protocolo MQTT: após pesquisas do funcionamento do protocolo, foi notado a grande utilidade em casos de comunicação M2M;
- Sensores para medir temperatura: inicialmente o projeto usaria o sensor DHT11 para obter umidade e temperatura. Após comparações com o LM35, esta ideia foi abolida, visto que a temperatura obtida através do LM35 é mais precisa, o que pode ser um fator crucial dependendo da situação, como por exemplo, em CPDs onde a temperatura pode se elevar rapidamente.

6 CRONOGRAMA

A tabela 3 abaixo representa o cronograma a ser seguido para o presente trabalho. A letra (C) representa o que foi concluído, a letra (P) o que esta em progresso e a letra (N), por final, o que ainda não foi feito. .

Tabela 3: Cronograma

Atividades	TCC 1					TCC 2			
	Mar	Abr	Mai	Jun	Jul	Ago	Set	Out	Nov
1. Revisão dos apontamentos da banca		C							
2. Revisão bibliográfica	P	P	P	P	P	P	P	P	P
3. Redação do projeto de TCC			C	C					
4. Defesa do projeto de TCC					C				
5. Escrita da Monografia de TCC					N	N	N	N	N
6. Desenvolvimento do sistema					N	N	N		
7. Testes em ambiente real								N	N
8. Elaboração da apresentação final								N	N
9. Defesa final do TCC									N

REFERÊNCIAS

ACTIVEMQ. **Apollo: ActiveMQ's next generation of messaging**. 2016. Acesso em: 10 mai. 2016. Disponível em: <https://activemq.apache.org/apollo/>.

ADRIEL, W. **Introdução ao Framework PHP Laravel**. 2015. Acesso em: 19 mai. 2016. Disponível em: <http://www.devmedia.com.br/introducao-ao-framework-php-laravel/33173>.

AGRIYA. **Pros and Cons of most Favoured PHP Frameworks**. 2015. Acesso em: 07 jun. 2016. Disponível em: <https://www.agriya.com/blog/2015/08/31/pros-cons-favoured-php-frameworks>.

ALIEXPRESS. **13 em 1 Starter Kit Módulos de Sensores de Chama para Arduino + MCU Uso Educacional**. 2016. Acesso em: 04 jun. 2016. Disponível em: <http://pt.aliexpress.com/item/13-in-1-Sensors-Modules-Starter-Kit-Flame-for-Arduino-MCU-Education-User-TE272/32572750949.html?spm=2114.42010708.4.4.uRa5cv>.

ALVAREZ, M. A. **Breve história do PHP**. jul 2004. Acesso em: 19 mai. 2016.

ARDUINO. **Arduino Ethernet Shield**. 2016. Acesso em: 07 abr. 2016. Disponível em: <https://www.arduino.cc/en/Main/ArduinoEthernetShield>.

ARDUINO. **Arduino Mega**. 2016. Acesso em: 07 abr. 2016. Disponível em: <https://www.arduino.cc/en/Main/ArduinoBoardMega>.

ARDUINO. **Arduino Shields**. 2016. Acesso em: 07 abr. 2016. Disponível em: <https://www.arduino.cc/en/Main/ArduinoShields>.

ARDUINO. **Arduino Software**. 2016. Disponível em: <https://www.arduino.cc/en/Main/Software>.

ARDUINO. **Arduino UNO & Genuino UNO**. 2016. Acesso em: 07 abr. 2016. Disponível em: <https://www.arduino.cc/en/Main/ArduinoBoardUno>.

ARDUINO. **Arduino Yún**. 2016. Acesso em: 07 abr. 2016. Disponível em: <https://www.arduino.cc/en/Main/ArduinoBoardYun>.

ARDUINO. **Introduction: What is Arduino?** 2016. Acesso em: 07 abr. 2016. Disponível em: <https://www.arduino.cc/en/Guide/Introduction>.

ARDUINO. **Referência da Linguagem**. 2016. Disponível em: <http://playground.arduino.cc/Portugues/Referencia>.

ARDUINOECIA. **Sensor de umidade e temperatura DHT11**. 2013. Acesso em: 03 jun. 2016. Disponível em: <http://www.arduinoecia.com.br/2013/05/sensor-de-umidade-e-temperatura-dht11.html>.

ASSOCIATION, T. I. techreport, **Telecommunications Infrastructure Standard for Data Centers**. apr 2005. Disponível em: <https://manuais.iessanclemente.net/images/9/9f/Tia942.pdf>.

BANAVAR, G. et al. An efficient multicast protocol for content-based publish-subscribe systems. In: **Distributed Computing Systems, 1999. Proceedings. 19th IEEE International Conference on**. [S.l.: s.n.], 1999. p. 262–272. ISSN 1063-6927.

BRITTO, R. A. R. C. C. P. J. A. **Contador de Passagens**. 2003. Acesso em: 08 abr. 2016. Disponível em: <http://www.gta.ufrj.br/grad/01_1/contador555/contador_de_passagem.htm>.

BROWN, K. **Desenvolva um sensor de temperatura pronto para nuvem com o Arduino Uno e o IBM IoT Foundation, Parte 2**. dec 2014. Acesso em: 20 mai. 2016. Disponível em: <http://www.ibm.com/developerworks/br/cloud/library/cl-bluemix-arduino-iot2>.

CUNHA, A. F. **O que são sistemas embarcados**. [S.l.]: Saber Eletrônica, 2007.

ELETRICAL4U. **Light Dependent Resistor — LDR and Working Principle of LDR**. 2016. Acesso em: 28 mai. 2016. Disponível em: <http://www.electrical4u.com/light-dependent-resistor-ldr-working-principle-of-ldr/>.

EUROTECH, I. B. M. C. I. **MQTT V3.1 Protocol Specification**. [S.l.], 2010. Acesso em: 25 mai. 2016. Disponível em: <http://public.dhe.ibm.com/software/dw/webservices/ws-mqtt/mqtt-v3r1.html>.

EVANS, D. The internet of things: How the next evolution of the internet is changing everything. **Cisco Internet Business Solutions Group**, p. 11, 2011. Acesso em: 30 mar. 2016. Disponível em: <http://www.cisco.com/c/dam/en_us/about/ac79/docs/innov/IoT_IBSG_0411FINAL.pdf>.

FILIPEFLOP. **COMUNICAÇÃO PELA REDE COM O MÓDULO ETHERNET ENC28J60**. 2016. Acesso em: 04 jun. 2016. Disponível em: <http://blog.filipeflop.com/modulos/modulo-ethernet-enc28j60-arduino.html>.

FUCTURA. **Vantagens e desvantagens do PHP**. 2016. Acesso em: 25 mai. 2016. Disponível em: <http://www.fuctura.com.br/2013/01/vantagens-e-desvantagens-do-php/>.

GILMORE, W. J. **Dominando PHP e MySQL: Do iniciante ao profissional**. Rio de Janeiro: Alta Books editora, 2008.

HUNKELER, U.; TRUONG, H. L.; STANFORD-CLARK, A. Mqtt-s; a publish/subscribe protocol for wireless sensor networks. In: **Communication Systems Software and Middleware and Workshops, 2008. COMSWARE 2008. 3rd International Conference on**. [S.l.: s.n.], 2008. p. 791–798.

HW-GROUP. **Poseidon2 3268 - Remote I/O and sensor monitoring system**. 2016. Acesso em: 08 abr. 2016. Disponível em: <http://www.hw-group.com/download/Poseidon2-3268_FL_en.pdf>.

HW-GROUP. **Poseidon2 4002**. 2016. Acesso em: 08 abr. 2016. Disponível em: <http://www.hw-group.com/products/poseidon/poseidon2_4002_en.html>.

HW-GROUP. **Poseidon2 4002 Online Prague**. 2016. Acesso em: 01 jun. 2016. Disponível em: <http://poseidon2-4002.hwg.cz/>.

KANNAN, S. **Paho Python client for MQTT and G-code Visualization Talks, Chennaipy**. 2015. Acesso em: 04 jun. 2016. Disponível em: <http://www.shakthimaan.com/posts/2015/10/13/chennai-py-talks/news.html>.

KITRONIK. **How an LDR works**. 2014. Disponível em: <https://www.kitronik.co.uk/pdf/How_an_LDR_works.pdf>.

LEE, S. et al. Correlation analysis of mqtt loss and delay according to qos level. In: **The International Conference on Information Networking 2013 (ICOIN)**. [S.l.: s.n.], 2013. p. 714–717. ISSN 1550-445X.

LIMA, G. F. Controle de temperatura de um sistema de baixo custo utilizando a placa arduino. In: **IX Congresso de iniciação científica do IFRN**. [s.n.], 2013. Acesso em: 01 mai. 2016. Disponível em: <http://www2.ifrn.edu.br/ocs/index.php/congic/ix/paper/viewFile/765/134>.

MCROBERTS, M. **Arduino Básico**. [S.l.]: Novatec Editora, 2010.

MILANI, A. **MySQL - Guia do Programador**. [S.l.]: Novatec Editora, 2007. ISBN 9788575221037.

MODTRONIX. **DHT11 Temperature and Humidity Sensor Module**. 2016. Acesso em: 04 jun. 2016. Disponível em: <http://modtronix.com/mod-dht11.html>.

MUKHOPADHYAY, S. C. **Internet of Things: Challenges and Opportunities**. 2014. ed. [S.l.]: Springer, 2014. (Smart Sensors, Measurement and Instrumentation, 9).

MYSQL. **MySQL: The world's most popular open source database**. 2016. Acesso em: 27 mai. 2016. Disponível em: <mysql.com>.

NATIONALSEMICONDUCTOR. **LM35: Precision Centigrade Temperature Sensors**. [S.l.], 2000. Acesso em: 28 mai. 2016. Disponível em: <http://www.webtronico.com/documentos/LM35.pdf>.

NINJAGECKO. **MEASURING LIGHT USING YOUR PI**. 2016. Acesso em: 03 jun. 2016. Disponível em: <http://www.ninjagecko.co.uk/how-to-measure-the-amount-of-light-in-a-room-using-a-light-dependent-resistor/>.

OPENSOURCE. **What is open source ?** 2016. Acesso em: 07 abr. 2016. Disponível em: <https://opensource.com/resources/what-open-source>.

PACIEVITCH, Y. **HTML**. 2016. Acesso em: 27 mai. 2016. Disponível em: <http://www.infoescola.com/informatica/html/>.

PARSKO, L. F. **TUTORIAL Aplicações, Funcionamento e Utilização de Sensores**. 2006. Acesso em: 27 mai. 2016. Disponível em: <http://www.maxwellbohr.com.br/downloads/robotica/mec1000_kdr5000/tutorial_eletronica_-_aplicacoes_e_funcionamento_de_sensores.pdf>.

REYES, J. **15 Wonderfully Creative Uses for PHP**. 2009. Acesso em: 20 mai. 2016. Disponível em: <http://code.tutsplus.com/tutorials/15-wonderfully-creative-uses-for-php–net-4714>.

RFM. **Agriculture: Environmental Monitoring and Control in Poultry Farming**. 2016. Acesso em: 06 jun 2016. Disponível em: <http://wireless.murata.com/media/products/applications/DNT24_PoultryFarming.pdf>.

SADDAM. **Arduino Based Digital Thermometer**. 2015. Acesso em: 05 jun. 2016. Disponível em: <http://circuitdigest.com/microcontroller-projects/digital-thermometer-using-arduino>.

SCHIMIGUEL, P. T. B. J. Desenvolvimento de aplicações mobile cross-platform utilizando phonegap. **Revista Observatorio da Economia Latinoamericana**, 2016. Disponível em: <http://www.eumed.net/cursecon/ecolat/br/16/phonegap.html>.

SHELDON, A. O. R. **SQL: Um guia para iniciantes**. 3. ed. Rio de Janeiro: Editora ciência moderna Ltda., 2009.

SILVA, M. S. **Introdução às CSS**. 2003. 10/12/2003. Acesso em: 28 mai. 2016. Disponível em: <http://www.maujor.com/tutorial/intrtut.php>.

SOARES, W. **PHP5: Conceitos, Programação e Integração com Banco de Dados**. 6. ed. São Paulo: Editora Érica Ltda., 2010.

STEFFENS, C. A. **O funcionamento e uso de alguns sensores usando a placa de som de um PC**. 2016. Aquisição de dados em laboratório didático de física. Acesso em: 08 abr. 2016. Disponível em: <http://www.if.ufrgs.br/mpef/mef004/20061/Cesar/SENSORES-Definicao.html>.

SUNROM. **DHT11: Humidity and Temperature Sensor**. [S.l.], 2012. Acesso em: 02 jun. 2016. Disponível em: <http://robocraft.ru/files/datasheet/DHT11.pdf>.

TELEGRAM. **Bots: An introduction for developers**. 2016. Acesso em: 03 jun. 2016. Disponível em: <https://core.telegram.org/bots>.

TELEGRAM. **Telegram FAQ**. 2016. Acesso em: 03 jun. 2016. Disponível em: <https://telegram.org/faq>.

TIOBE. **TIOBE Index for June 2016**. 2016. Acesso em: 01 jun. 2016. Disponível em: <http://www.tiobe.com/tiobe_index?page=index>.

TUTORIALSPOINT. **SQL TUTORIAL**. 2016. Acesso em: 03 jun. 2016. Disponível em: <http://www.tutorialspoint.com/sql/sql_tutorial.pdf>.

W3SCHOOLS. **CSS Tutorial**. 2016. Acesso em: 01 jun. 2016. Disponível em: <http://www.w3schools.com/css/>.

W3SCHOOLS. **HTML5 Introduction**. 2016. Acesso em: 01 jun. 2016. Disponível em: <http://www.w3schools.com/html/html5_intro.asp>.

XANDó, F. **PRTG - monitorando totalmente sua rede e infraestrutura**. 2012. Acesso em: 08 abr. 2016. Disponível em: <http://www.fxreview.com.br/2012/11/prtg-monitorando-totalmente-sua-rede-e.html>.

XDSOFTWARE. **Apostila de SQL**. 2016. Acesso em: 20 mai. 2016. Disponível em: <http://www.xdsoftware.com/demo/BR-pt/manuais/ManSQL1.pdf>.